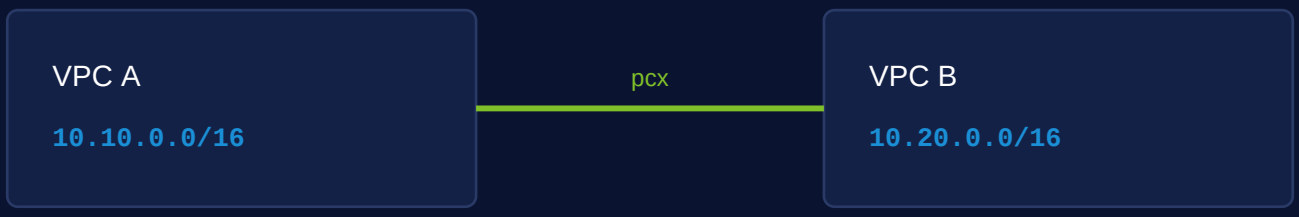
AWS NETWORKING / PROJECT 01

VPC Peering

Connecting two VPCs and proving private connectivity

Project report: EC2, custom route tables, security groups and a peering connection



SUBMITTED BY

Jayendra Singh

Author: Jayendra Singh | B.Tech Information Technology, BIET Jhansi

Programme: EWB Courses, AWS Architect Associate (Project 01, Oct 2026)

Region: Asia Pacific (Sydney), ap-southeast-2 | Completed: 18 October 2026

LEARN. ELEVATE. EMPOWER.

1. Objective

The goal of this project was to connect two separate AWS Virtual Private Clouds (VPCs) with a VPC peering connection so that workloads in each VPC can talk to each other over private IPv4 addresses, without using the public internet. The build also had to show that each route and security rule has a purpose: non-overlapping CIDR ranges, explicit route table associations, forward and return routes, and tightly scoped inbound rules.

What this report demonstrates:

- Planning non-overlapping VPC and subnet CIDR ranges.
- Creating public and private subnets with explicit route table associations.
- Requesting, accepting and activating a peering connection, then adding routes on both sides.
- Launching EC2 instances in the two VPCs and verifying private connectivity.

2. Architecture and address plan

VPC A (`10.10.0.0/16`) holds a public and a private subnet and is the only VPC with an internet gateway. VPC B (`10.20.0.0/16`) has a single private subnet and no internet gateway. A peering connection links the two VPCs, so the private subnets reach each other directly using private IP addresses.

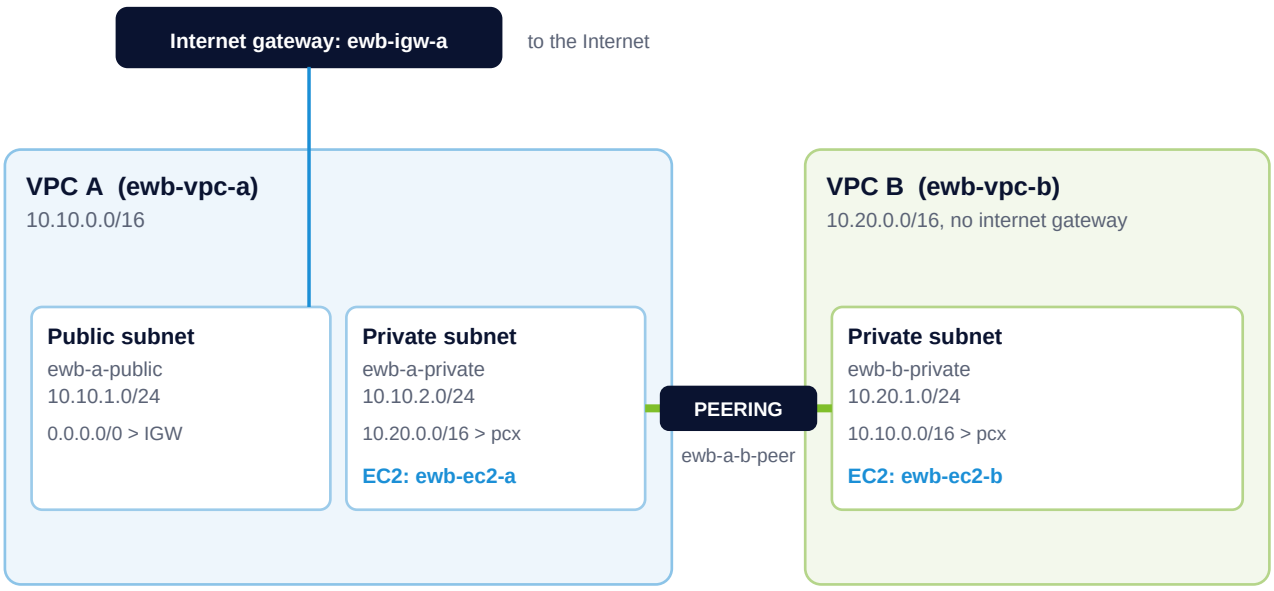


Figure 1. Architecture of the project (conceptual). Peering does not give VPC B internet access through VPC A.

Address plan

Resource (Name tag)	Placement	IPv4 CIDR
ewb-vpc-a	VPC A	`10.10.0.0/16`
ewb-a-public	VPC A, public subnet	`10.10.1.0/24`
ewb-a-private	VPC A, private subnet	`10.10.2.0/24`
ewb-vpc-b	VPC B	`10.20.0.0/16`
ewb-b-private	VPC B, private subnet	`10.20.1.0/24`

The two VPC ranges do not overlap, which is a hard requirement: AWS refuses to create a peering connection between VPCs with overlapping CIDRs. Every /24 subnet sits inside its own VPC range.

3. Resource inventory

The table lists the resources created in the **Asia Pacific (Sydney) ap-southeast-2** Region, with the IDs shown in the AWS console.

Resource	Name	ID / detail
VPC A	ewb-vpc-a	`vpc-0e13b1275c4404f86 (10.10.0.0/16)`
VPC B	ewb-vpc-b	`vpc-0b570eaf09596cbfa (10.20.0.0/16)`
Subnet	ewb-a-public	`subnet-093184ce1cf20194c (10.10.1.0/24)`
Subnet	ewb-a-private	`subnet-0c3c48d885ebe... (10.10.2.0/24)`
Subnet	ewb-b-private	`subnet-09aac6debd58277e4 (10.20.1.0/24)`
Internet gateway	ewb-igw-a	`igw-083541cabc1ddaac6, attached to ewb-vpc-a`
Route table	ewb-rt-a-public	`rtb-0f1d18e9fea198201, associated with ewb-a-public`
Route table	ewb-rt-a-private	`Created in VPC A for ewb-a-private`
Route table	ewb-rt-b-private	`rtb-0a50e327a9e62155e, associated with ewb-b-private`
Peering connection	ewb-a-b-peer	`pcx-0ee8e4be0ede9ba99, status Active`
EC2 instance	ewb-ec2-a	`i-0b2fa2f3b7306a262, t3.micro, ap-southeast-2a`
EC2 instance	ewb-ec2-b	`i-08a657c4b1065e30b, t3.micro, ap-southeast-2a`

4. Implementation

Step 1: Create VPC A

In the VPC console I chose *Create VPC* with **VPC only**, so that every subnet and route could be controlled manually. The VPC was named `ewb-vpc-a` with the IPv4 CIDR `10.10.0.0/16`, no IPv6 CIDR and default tenancy. The VPC reached the Available state.

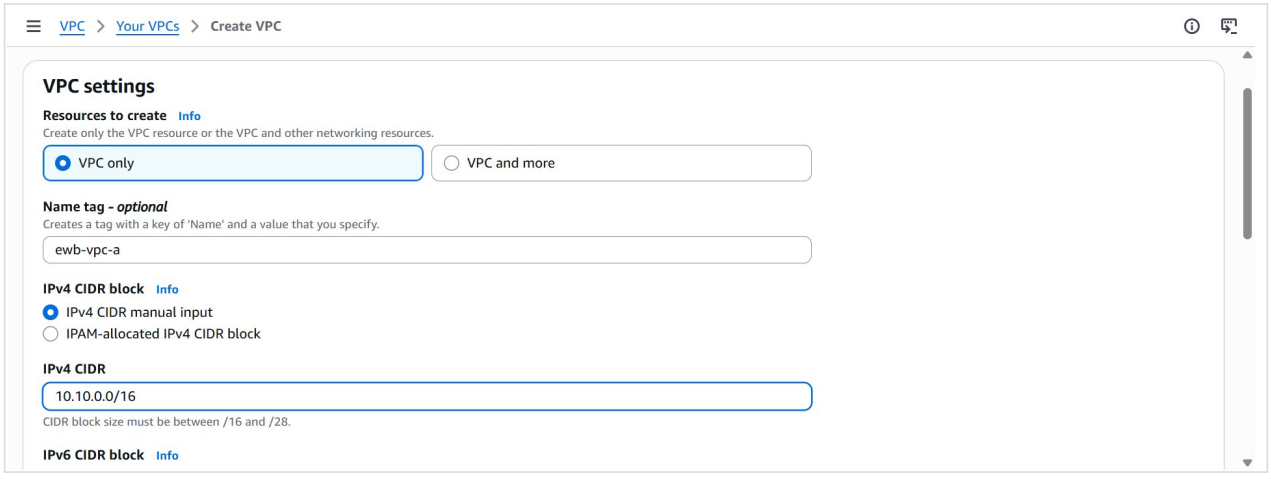


Figure 2. Create VPC wizard: VPC only, name ewb-vpc-a, IPv4 CIDR 10.10.0.0/16.

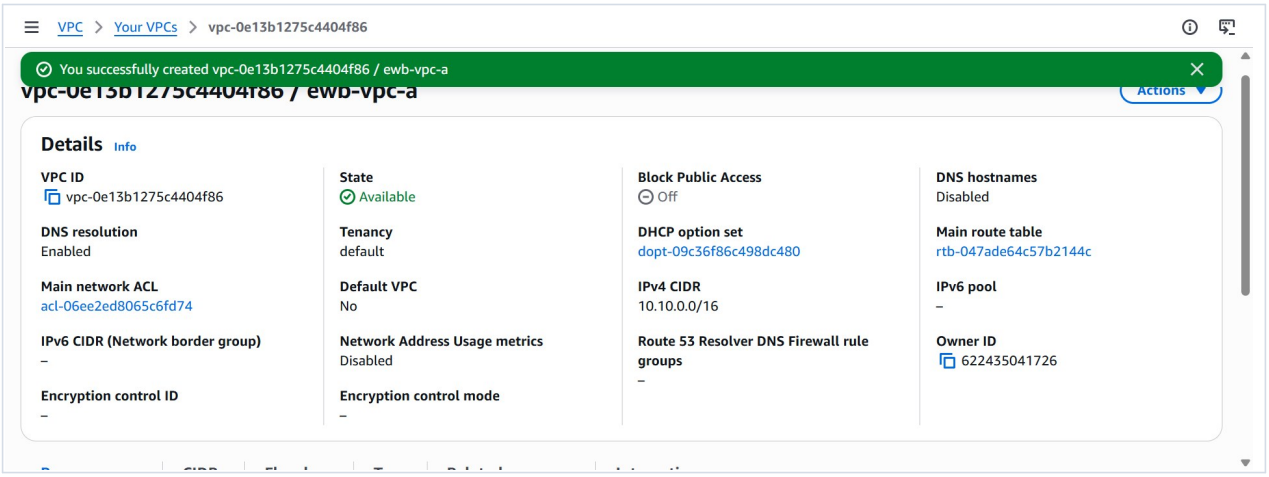


Figure 3. VPC ewb-vpc-a (vpc-0e13b1275c4404f86) created successfully and in the Available state.

Step 2: Create the subnets in VPC A

Two subnets were created in VPC A: `ewb-a-public` (10.10.1.0/24) and `ewb-a-private` (10.10.2.0/24). The subnet associations screen in Figure 7 lists both subnets with their CIDRs.

Step 3: Attach the internet gateway

An internet gateway named `ewb-igw-a` was created and attached to ewb-vpc-a. An internet gateway belongs to the VPC and is used as a route target in the route table, so attaching it alone does not give a subnet internet access.

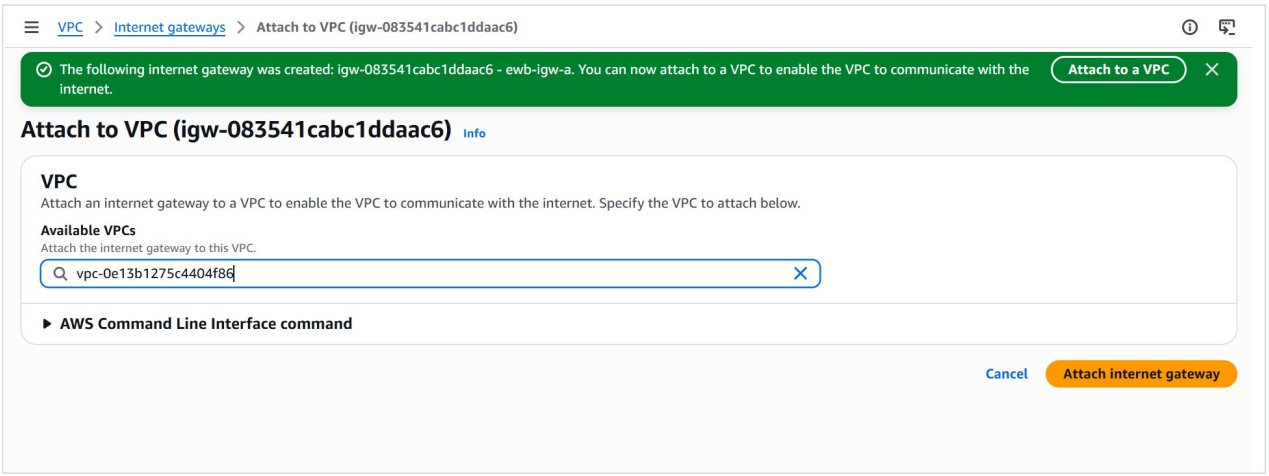


Figure 4. Attaching internet gateway ewb-igw-a (igw-083541cabc1ddaac6) to vpc-0e13b1275c4404f86.

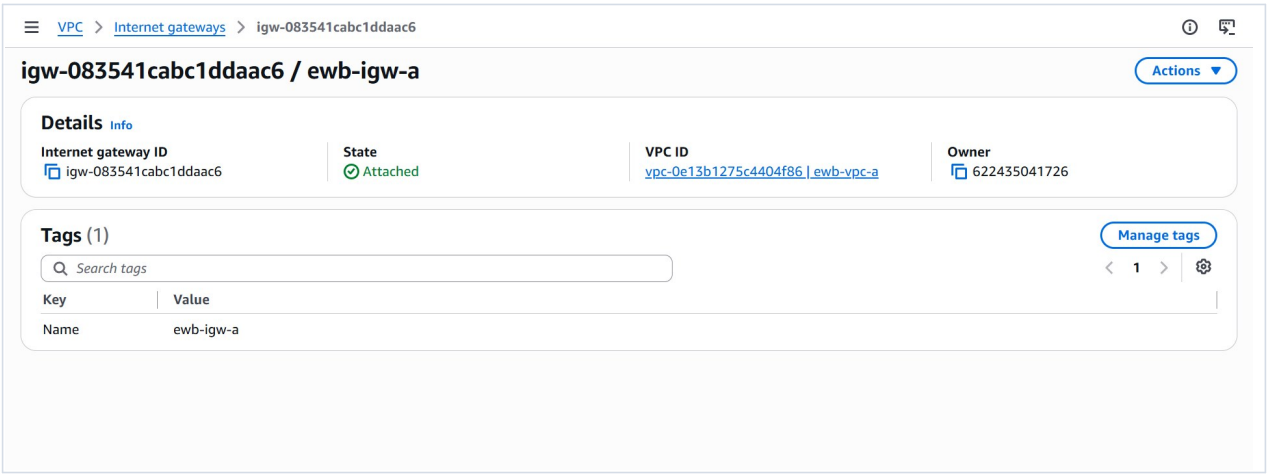


Figure 5. Internet gateway ewb-igw-a is in the Attached state, linked to ewb-vpc-a.

Step 4: Create the route tables in VPC A

A custom route table `ewb-rt-a-public` was created for VPC A, and `ewb-a-public` was explicitly associated with it. A second route table, `ewb-rt-a-private`, was created for the private subnet. It keeps only the local route and has no internet default route, so the private subnet stays private.

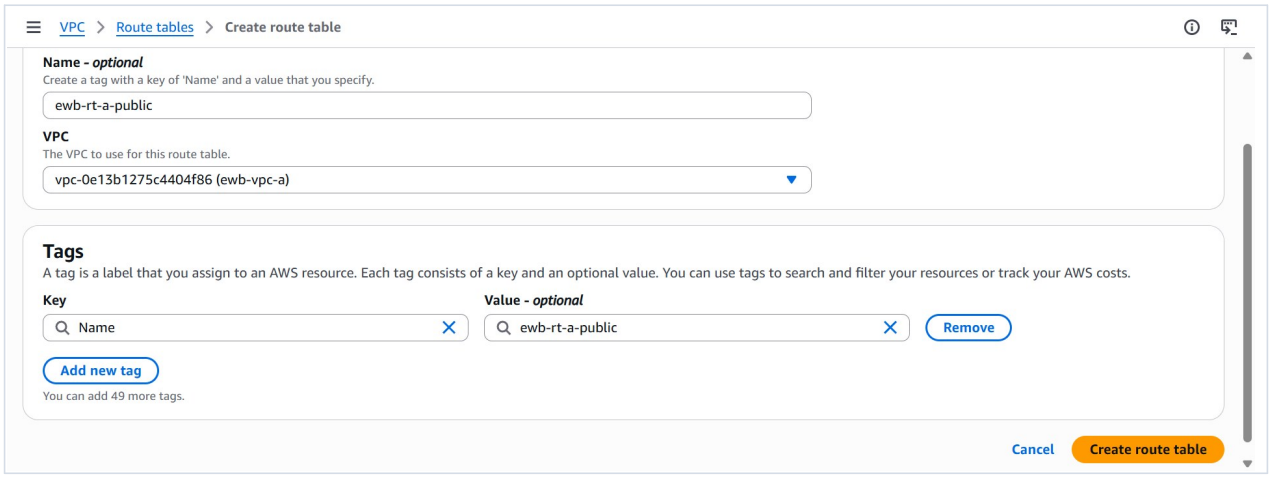


Figure 6. Creating route table ewb-rt-a-public in vpc-0e13b1275c4404f86 (ewb-vpc-a).

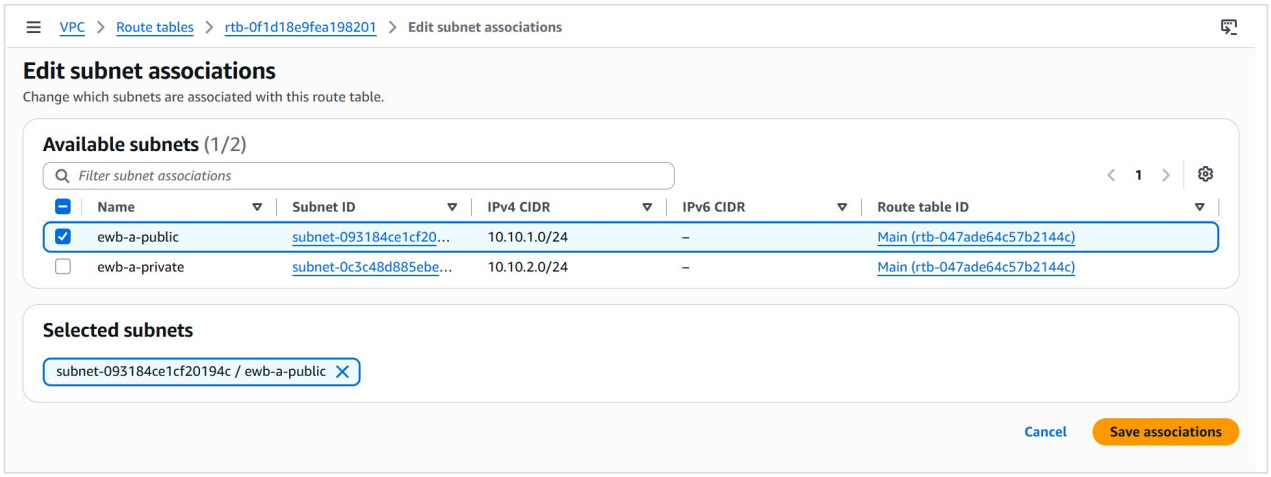


Figure 7. Explicit subnet association: ewb-a-public (10.10.1.0/24) selected for ewb-rt-a-public.

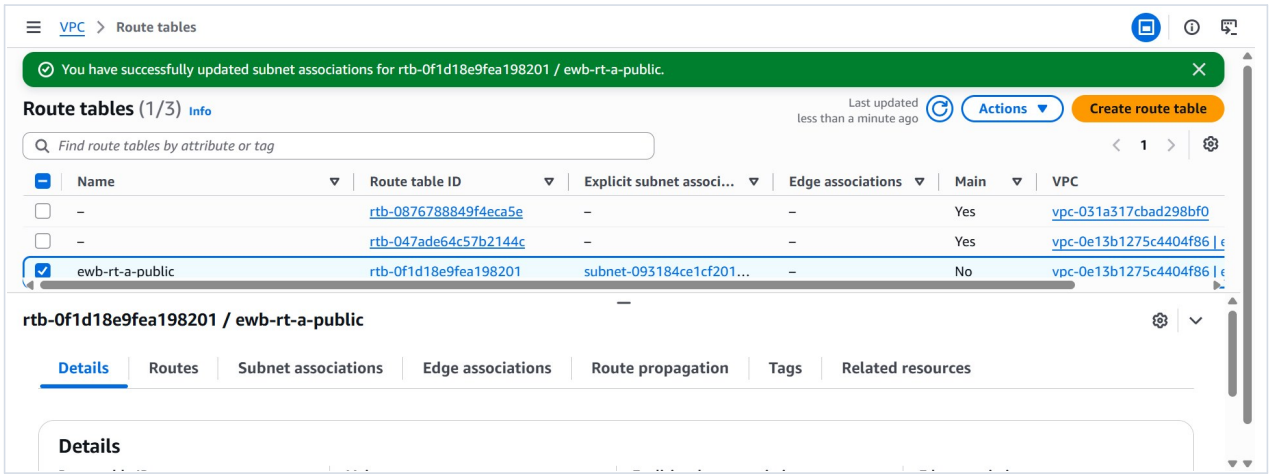


Figure 8. Subnet associations updated successfully; ewb-rt-a-public is now associated with subnet-093184ce1cf20...

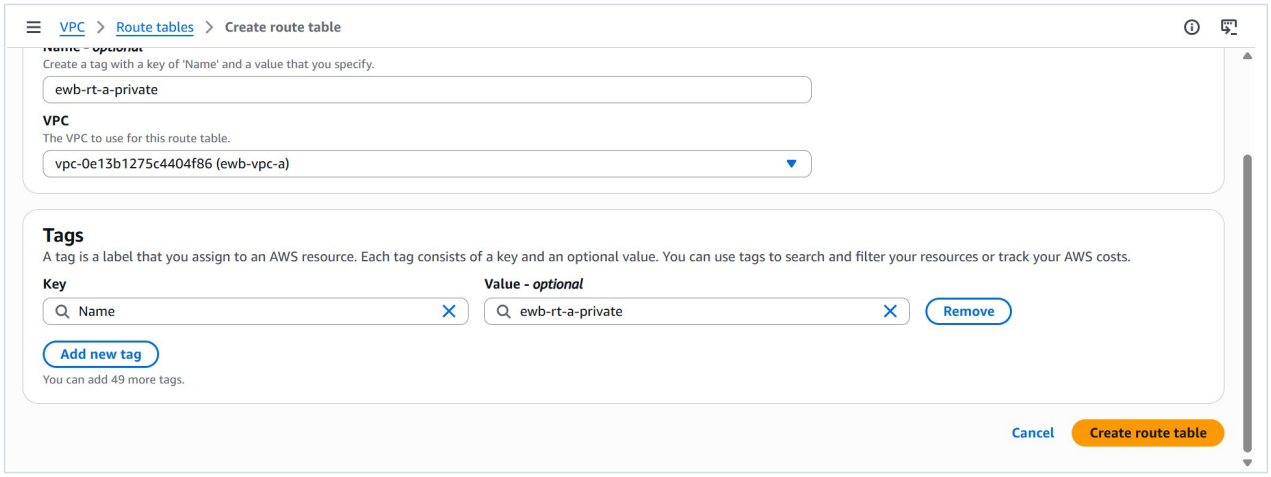


Figure 9. Creating route table ewb-rt-a-private in ewb-vpc-a.

Step 5: Create VPC B, its subnet and route table

VPC B (`ewb-vpc-b`, 10.20.0.0/16) was created with no internet gateway. Inside it, the private subnet `ewb-b-private` (10.20.1.0/24) and the route table `ewb-rt-b-private` were created, and the subnet was explicitly associated with the route table.

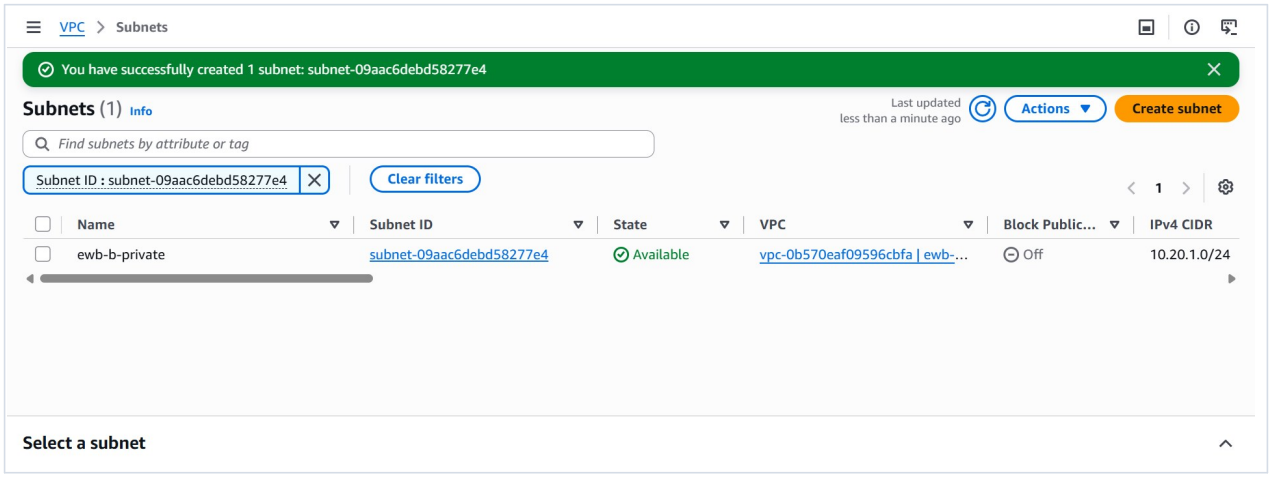


Figure 10. Subnet ewb-b-private (subnet-09aac6debd58277e4, 10.20.1.0/24) created and Available in VPC B.

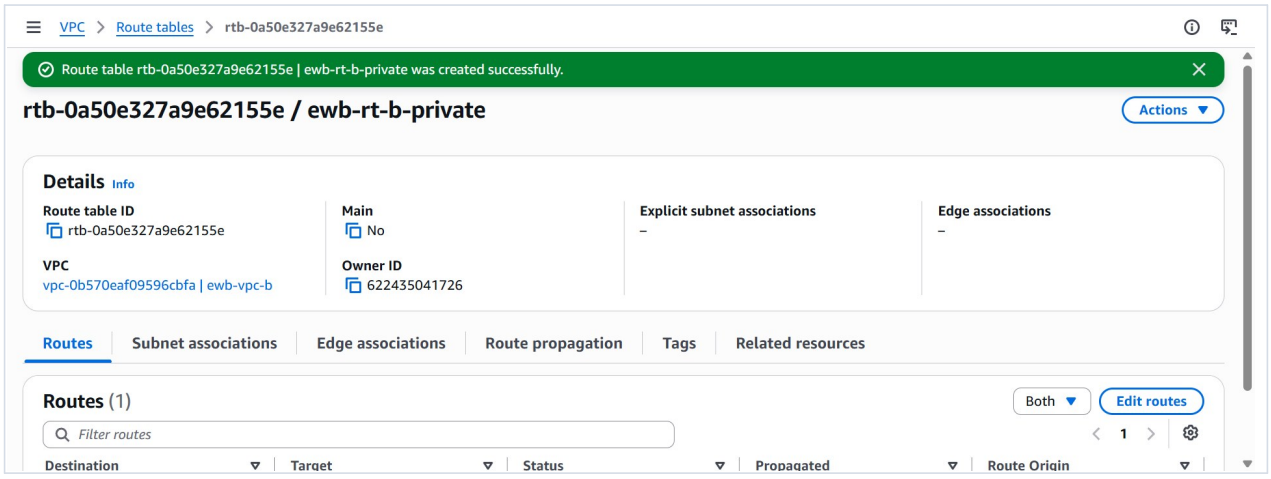


Figure 11. Route table ewb-rt-b-private (rtb-0a50e327a9e62155e) created in ewb-vpc-b.

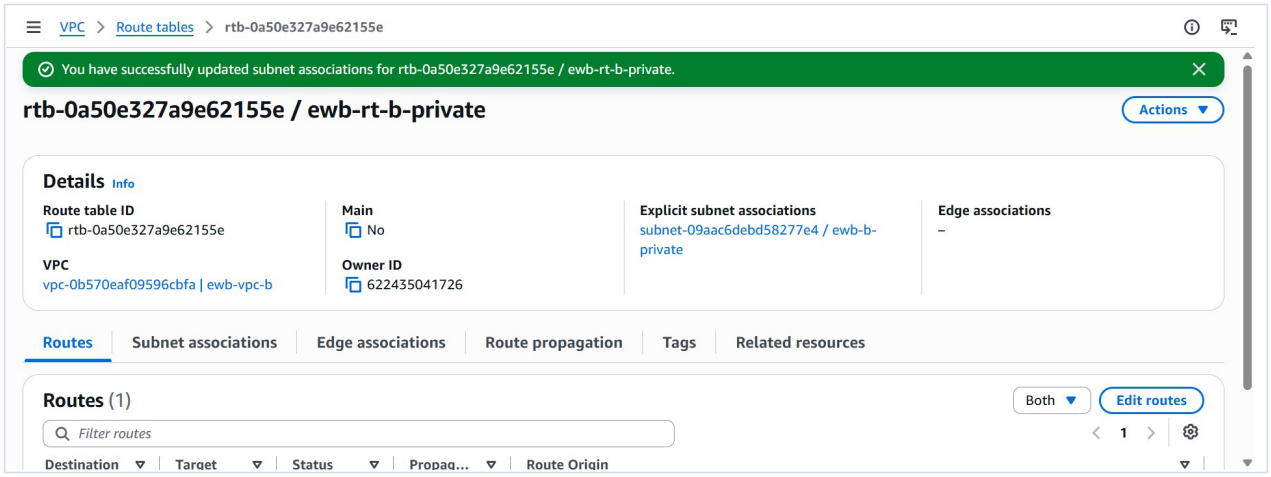


Figure 12. Subnet association updated: subnet-09aac6debd58277e4 / ewb-b-private is now explicitly associated with ewb-rt-b-private.

Step 6: Request and accept the peering connection

A peering connection named `ewb-a-b-peer` was requested with ewb-vpc-a as the requester and ewb-vpc-b as the accepter. Both VPCs are in the same account and Region. The request first showed *Pending acceptance*. Even within one account, the request must be accepted manually. After acceptance the status changed to **Active**, and the requester CIDR (10.10.0.0/16) and accepter CIDR (10.20.0.0/16) were both shown.

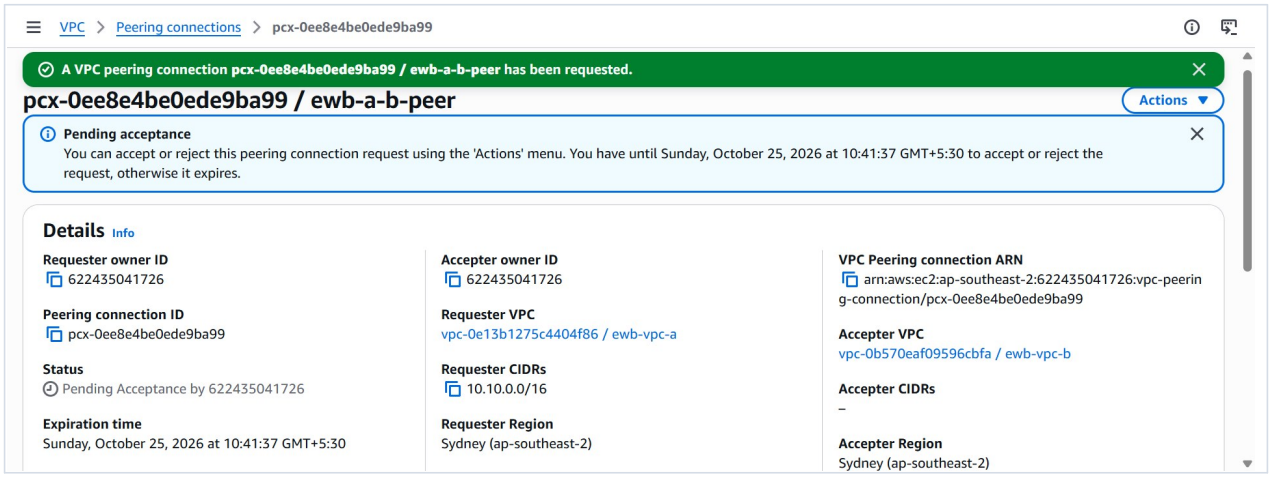


Figure 13. Peering connection pcx-0ee8e4be0ede9ba99 (ewb-a-b-peer) requested: status Pending acceptance.

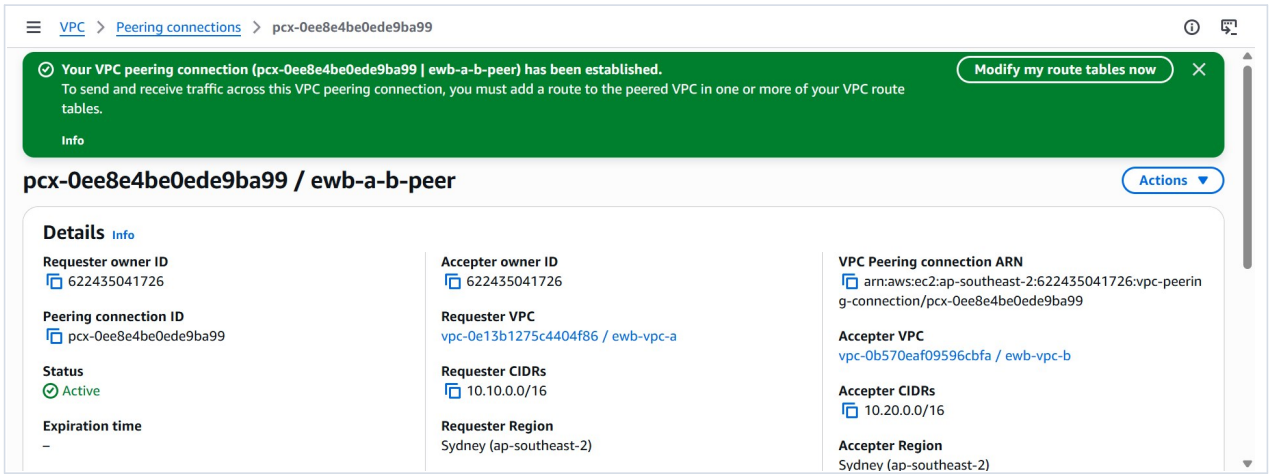


Figure 14. Peering connection ewb-a-b-peer established and Active, with requester 10.10.0.0/16 and accepter 10.20.0.0/16.

> **Important:** an Active peering connection does not by itself let traffic flow. It only creates the link. Routes in both VPCs and the security group rules decide whether traffic can use it.

Step 7: Add routes on both sides

Each side needs a route to the other VPC, because a reply needs a way back as well as a way out. The routes below were configured, with `pcx-0ee8e4be0ede9ba99` as the target for the peer CIDR.

Route table	Destination	Target	Purpose
ewb-rt-a-private	`10.10.0.0/16`	local	VPC A internal traffic
ewb-rt-a-private	`10.20.0.0/16`	pcx (ewb-a-b-peer)	Forward path from VPC A to VPC B
ewb-rt-b-private	`10.20.0.0/16`	local	VPC B internal traffic
ewb-rt-b-private	`10.10.0.0/16`	pcx (ewb-a-b-peer)	Return path from VPC B to VPC A
ewb-rt-a-public	`10.10.0.0/16`	local	VPC A internal traffic
ewb-rt-a-public	`0.0.0.0/0`	igw (ewb-igw-a)	Internet access for the public subnet
ewb-rt-a-public	`10.20.0.0/16`	pcx (ewb-a-b-peer)	Public subnet to VPC B

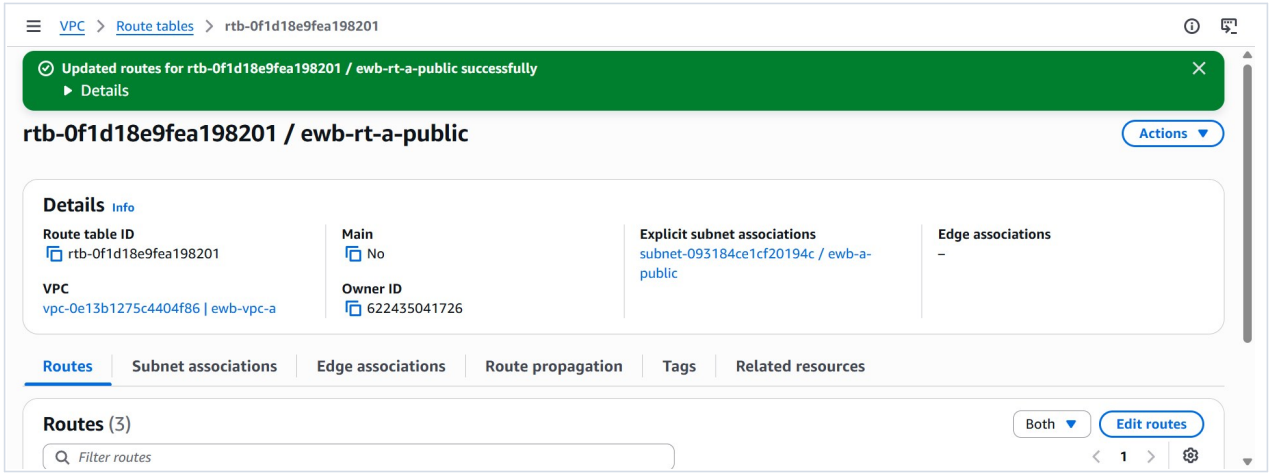


Figure 15. Routes updated successfully for ewb-rt-a-public (rtb-0f1d18e9fea198201); the route table now has 3 routes.

The more specific `10.20.0.0/16` route is chosen for traffic to VPC B instead of the broader internet default route, because AWS always picks the most specific matching route.

Step 8: Launch the EC2 instances

Two `t3.micro` instances were launched in Availability Zone ap-southeast-2a: `ewb-ec2-a` for VPC A and `ewb-ec2-b` for VPC B. Both reached the Running state and passed all 3 of 3 status checks.

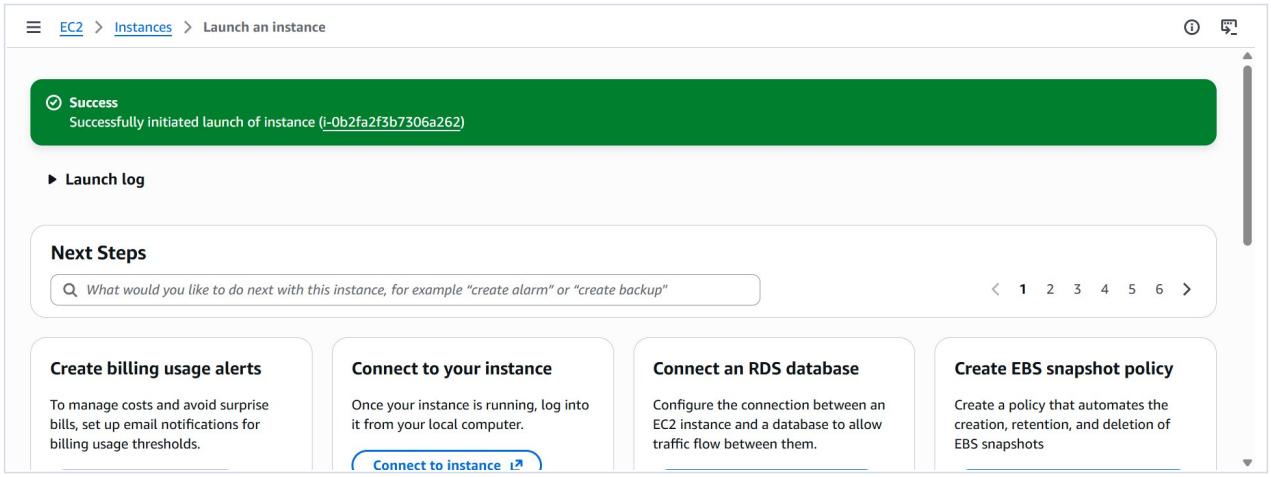


Figure 16. EC2 launch initiated successfully for instance i-0b2fa2f3b7306a262.

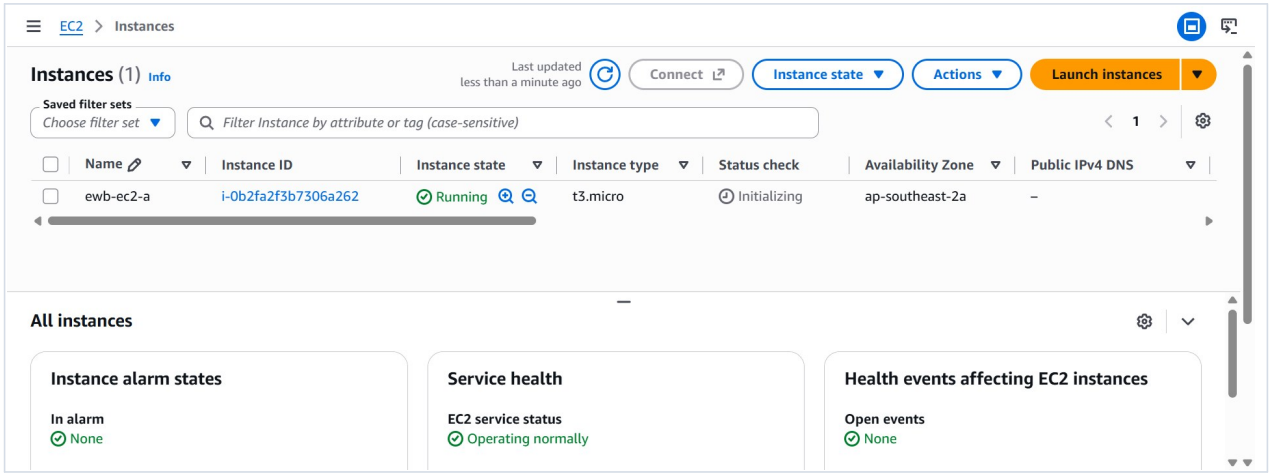


Figure 17. ewb-ec2-a (i-0b2fa2f3b7306a262) running; status checks initializing.

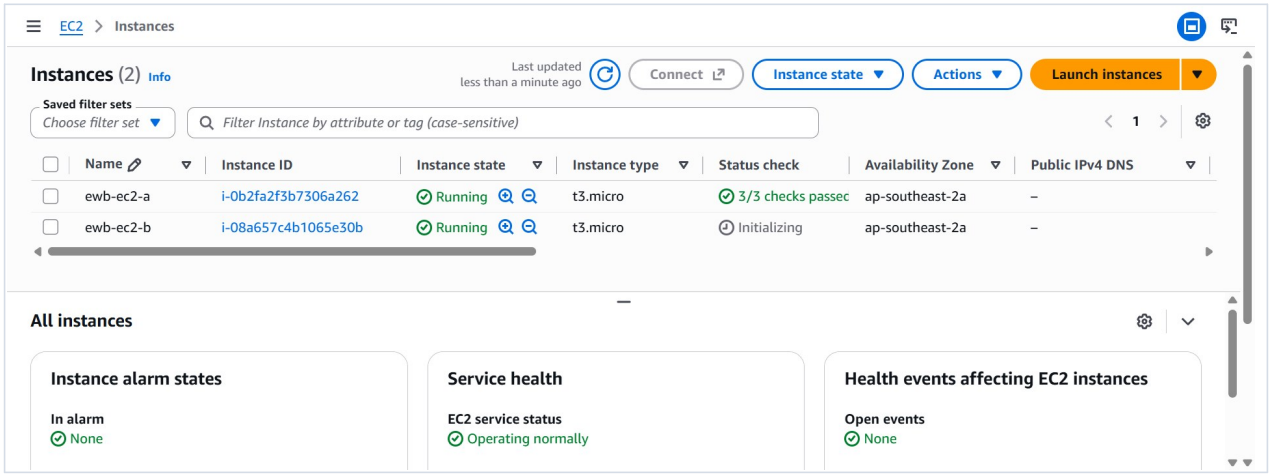


Figure 18. Second instance ewb-ec2-b (i-08a657c4b1065e30b) launched; ewb-ec2-a now shows 3/3 checks passed.

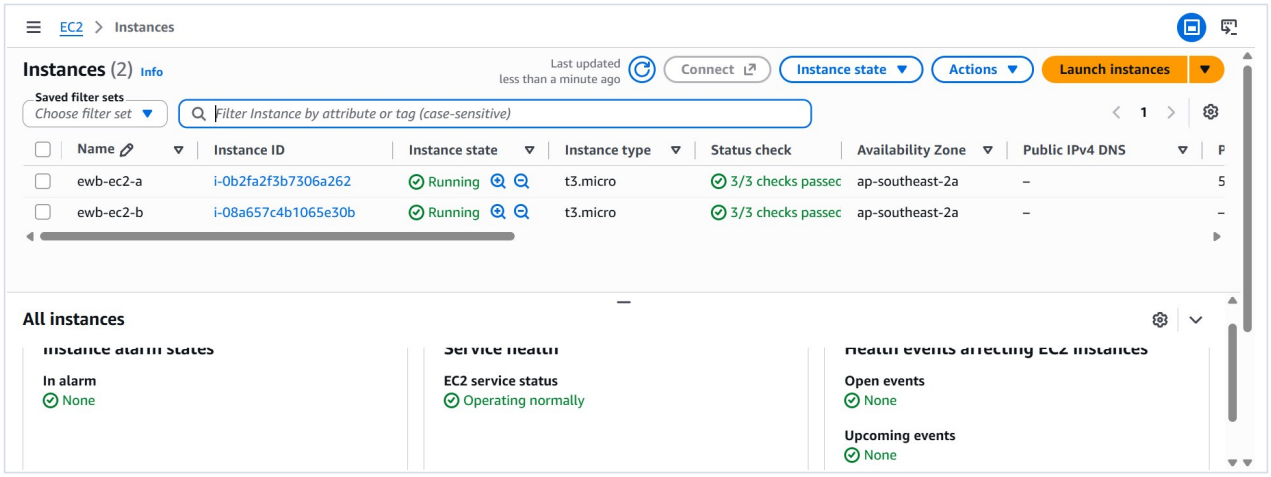


Figure 19. Both instances Running with 3/3 status checks passed, ready for connectivity testing.

5. Security design

Security groups act as stateful firewalls, so replies to permitted traffic are allowed automatically. The inbound rules follow the principle of least privilege: only the specific lab hosts are allowed, and nothing is opened to `0.0.0.0/0`.

Security group	Type / port	Source
ewb-sg-app-a (VPC A)	Custom ICMP IPv4, Echo request	Private IP of the instance in VPC B (/32)
ewb-sg-app-a (VPC A)	SSH / TCP 22	Private IP of the administration host (/32)
ewb-sg-app-b (VPC B)	Custom ICMP IPv4, Echo request	Private IP of the instance in VPC A (/32)
ewb-sg-app-b (VPC B)	SSH / TCP 22	Private IP of the administration host (/32)

Both application instances are private: they have no public IPv4 address, and the subnets are not routed to an internet gateway except the public subnet in VPC A. The key pair stays on my own computer and is never copied to any instance.

6. Testing and verification

Peering is considered successful only if traffic actually flows, so the tests below check both directions and also include a controlled failure. All tests use private destination IP addresses.

Test	Where / command	Expected result
01 Baseline	ewb-ec2-a, before peering routes: `ping -c 4 <private IP of ewb-ec2-b>`	No replies
02 Forward path	ewb-ec2-a, after routes: `ping -c 4 <private IP of ewb-ec2-b>`	4 replies, 0% packet loss
03 Reverse path	ewb-ec2-b, after routes: `ping -c 4 <private IP of ewb-ec2-a>`	4 replies, 0% packet loss
04 Private SSH	SSH to ewb-ec2-b through the bastion path	Shell on ewb-ec2-b
05 Isolation	EC2 instance details of both private hosts	No public IPv4 address
06 Controlled failure	Remove only the ICMP Echo rule on ewb-sg-app-b, ping again, then restore the exact rule	Replies stop, then resume

Explanation of the controlled failure: removing the Echo request rule changes only the security layer, while the route and peering stay in place. The ping fails, which proves that security groups are evaluated on top of

routing. Restoring the rule brings replies back, so the fault is isolated to that one rule.

7. Forward path and return path

Forward path. A packet from the instance in VPC A to a 10.20.x.x address matches the `10.20.0.0/16` route in ewb-rt-a-private, whose target is the peering connection. It crosses the peering link and arrives in VPC B, where the destination security group must allow the traffic.

Return path. The reply to a 10.10.x.x address matches the `10.10.0.0/16` route in ewb-rt-b-private, which also points at the peering connection. Without that route the reply would have nowhere to go, so a one-way route is not enough. The security group is stateful, so the reply needs no extra inbound rule.

8. Key concepts and questions answered

- **Why is the return route needed?** Routing is per direction. If only VPC A knows the way to VPC B, requests arrive but replies cannot get back.
- **Why does an Active peering connection not guarantee a successful ping?** Active only means the link exists. Route tables, security groups and network ACLs still decide whether traffic is allowed.
- **Can VPC B use the internet gateway of VPC A over peering?** No. Peering is not transitive and does not extend internet access, so VPC B stays private.
- **Why must the CIDR ranges not overlap?** Overlapping ranges would make routing ambiguous, so AWS blocks the peering connection.
- **Is peering transitive?** No. A-B plus A-C does not give B-C connectivity.

9. Troubleshooting checklist

Symptom	What to check
Ping fails after acceptance	Both private route tables, the correct pcx ID and Active state, the Echo request rule on the destination, and the actual host IPs.
Peering creation fails	Check every CIDR of both VPCs for overlap, including secondary CIDRs.
SSH times out	Instance state, destination IP, subnet route table, then the security group rule, in that order.
Permission denied (publickey)	Correct key pair, local identity file and the ec2-user name. Routing is working if SSH authentication was reached.

10. Cleanup plan

To avoid charges, all resources created for this lab should be removed once the evidence is saved:

- Terminate the EC2 instances and confirm their volumes are deleted. Release any Elastic IP and remove the unused key pair.
- Delete the peer routes and the peering connection ewb-a-b-peer.
- Delete the lab security groups, then the subnets and custom route tables.
- Detach and delete ewb-igw-a, then delete both VPCs.
- Verify with the project tag that nothing chargeable remains.

11. What I learned

This project showed me that a working network is the combination of several small pieces, and that each piece has one clear job. A VPC peering connection only creates the link. Routes on both sides send the traffic across it, and security groups decide which hosts may talk. I also learned why careful CIDR planning at the start matters, since overlapping ranges cannot be fixed later without rebuilding. Testing from both directions, and breaking one rule on purpose, made it much easier to understand where a connectivity problem can come from. The same ideas apply to many real AWS designs, such as connecting application and database environments privately.

Report prepared by **Jayendra Singh**.